

บทที่ 3

XML

3.1 บทนำ

XML หรือ Extensible Markup Language เป็นภาษาที่ถูกกำหนดขึ้นโดยกลุ่ม W3C (World Wide Web Consortium) โดยใช้ภาษา SGML (Generalized Markup Language) เป็นแม่แบบ มีจุดมุ่งหมายหลักเพื่อลดหรือกำจัดจุดด้อยของ HTML เอง

ลักษณะการใช้งาน XML นั้น มิได้ถูกใช้งานแทน HTML แต่อย่างใด ตรงกันข้ามการใช้งาน XML เองมักใช้ควบคู่ไปกับตัวภาษา HTML เนื่องจาก XML จะเน้นหนักไปที่ตัวของข้อมูลเป็นหลัก โดยจะเข้ามาควบคุมการจัดเก็บและลักษณะของตัวเนื้อข้อมูลเป็นสำคัญ ซึ่งต่างกับตัว HTML ที่จะเน้นหนักในเรื่องของการแสดงผล ดังนั้นโดยทั่วไปจึงมักใช้งาน XML ในเรื่องของการจัดการข้อมูล และใช้ HTML ในเรื่องของการแสดงผล

เนื่องจากการออกแบบ XML นั้นออกแบบตาม SGML ดังนั้นความสามารถหลักจึงมีความใกล้เคียงกับตัว SGML ด้วย คือสามารถออกแบบและสร้างรูปแบบของเอกสารได้เองตามลักษณะของการใช้งาน สามารถสร้างและกำหนดอีลีเมนต์เพิ่มขึ้นได้เอง นอกจากนี้ยังสามารถสร้างนิยามของอีลีเมนต์ที่กำหนดขึ้นมานั้นให้เข้าใจโดยทั่วกันได้อีกด้วย

และเนื่องจากตัว XML เองเป็นเฉพาะตัวของเนื้อข้อมูลไม่เกี่ยวกับการแสดงผลดังนั้นจึงทำให้สามารถออกแบบชิ้นงานได้ง่าย เนื่องจากในการทำงานจริงนั้นทำให้เราสามารถออกแบบการใช้งานกับการแสดงผลควบคู่กันไปได้เลย และการปรับแก้ตัวข้อมูลในภายหลังก็สามารถทำได้ง่ายเนื่องจากหากเราต้องการปรับแก้เฉพาะส่วนข้อมูลก็ไม่ต้องไปยุ่งกับส่วนของการแสดงผล หรือหากต้องการปรับแก้ในส่วนของการแสดงผลก็ไม่จำเป็นต้องไปยุ่งกับส่วนของตัวเนื้อข้อมูล ซึ่งจะช่วยลดปัญหาในด้านของการพัฒนาแอปพลิเคชันได้

ความถูกต้องของเอกสาร XML แบ่งออกเป็น 2 รูปแบบคือ

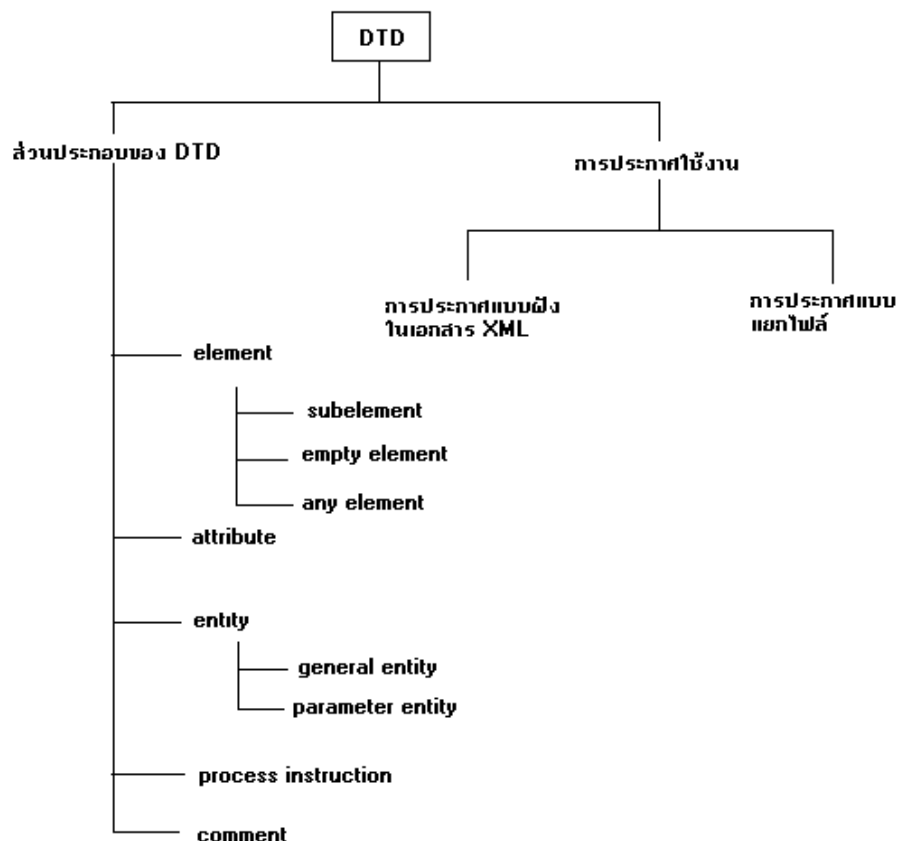
1. Well Formed XML หมายถึงเอกสาร XML ที่เขียนถูกต้องตาม syntax ของ XML ซึ่งได้แก่การปฏิบัติตามกฎของการซ้อนทับ (Nesting rule) การตั้งชื่ออีลีเมนต์ การเขียนแอตทริบิวต์ ฯลฯ
2. Valid XML โดยเอกสาร XML จะ valid ได้ก็ต่อเมื่อเอกสารนั้นมีการตรวจสอบกับนิยามของเอกสาร อันได้แก่ DTD (Document Type Definition) และ XML Schema อันจะได้กล่าวถึงในรายละเอียดต่อไป

3.2 Document Type Definition (DTD)

เนื่องจากการที่เราสามารถกำหนดรูปแบบเอกสาร XML ได้ด้วยตนเอง และสามารถสร้างอีลีเมนต์ของ XML ขึ้นใช้งานได้อย่างอิสระ ดังนั้นจึงต้องมีกลไกในการตรวจสอบความถูกต้องของอีลีเมนต์ภายในเอกสาร XML นั้นให้อยู่ในรูปแบบที่เราต้องการ กลไกในการตรวจสอบเอกสาร XML เราจะใช้ DTD เข้ามาช่วยในการตรวจสอบ

DTD เป็นการนิยามเอกสาร XML ให้เป็นเอกสารที่ถูกต้องสมบูรณ์ หรือ valid เป็นเสมือนการสร้างข้อตกลงให้กับเอกสาร XML ของเราว่า เอกสารจะต้องประกอบไปด้วยอีลีเมนต์อะไรได้บ้าง แต่ละอีลีเมนต์มีแอตทริบิวต์อะไร เป็นประเภทไหน ชนิดของตัวข้อมูลคืออะไร เป็นต้น ดังแสดงในรูปที่ 3-1 ทั้งนี้การใช้ DTD กำกับเอกสาร XML จะทำให้เอกสาร XML ของเรามีการใช้งานร่วมกับคนอื่นหรือแอปพลิเคชันอื่นได้ง่ายขึ้น โดยที่ผู้มาขอใช้เอกสารของเราก็ต้องมีการปฏิบัติตามข้อกำหนดที่ระบุไว้ใน DTD นั้นเอง

DTD ถือเป็นส่วนเพิ่มเติมของเอกสาร XML นั่นคือเราจะกำหนดให้เอกสาร XML ของเรามี DTD กำกับอยู่ด้วยหรือไม่ก็ได้ โดยเราจะกำหนดให้อยู่ภายในเอกสาร XML ของเราเลยหรือให้เป็นไฟล์แยกออกไปต่างหากก็ได้ ถ้าเรากำหนดให้ DTD แยกออกมาเป็นไฟล์ ต่างหากเราจะต้องสร้างไฟล์ ที่มีนามสกุลเป็น .dtd ขึ้นมาเพื่อเก็บรายละเอียดของ DTD



รูปที่ 3-1 แสดงองค์ประกอบภายใน DTD

ประโยชน์ของ DTD

จริง ๆ แล้ว DTD ไม่ใช่ข้อบังคับที่เอกสาร XML ทุกไฟล์จะต้องมีดังนั้นในบางกรณีการใช้หรือไม่ใช้ DTD ควบคู่ไปกับเอกสาร XML ก็จะได้ผลลัพธ์ที่ไม่แตกต่างกันเลย จึงเกิดคำถามตามมาว่าแล้วทำไมเราต้องใช้ DTD คำตอบก็คือหากงานหรือข้อมูลของเราเรียบง่ายไม่มีความซับซ้อน อีกทั้งเราเป็นผู้ใช้ข้อมูลแต่เพียงผู้เดียวแล้ว DTD ก็อาจไม่จำเป็นต้องมีก็ได้ แต่หากเราต้องการใช้เอกสารร่วมกับผู้อื่นแล้วตัว DTD นี้เองที่จะเป็นตัวนิยามข้อมูลเพื่อให้ผู้ใช้เอกสารมีความเข้าใจเอกสารตรงกัน

การที่เรากำหนดให้เอกสาร XML มี DTD จะทำให้เอกสารหรือข้อมูลของเราสื่อความหมายได้ดีขึ้น การแลกเปลี่ยนข้อมูลกับผู้อื่นก็ทำได้ง่าย ยกตัวอย่างเช่น เรามีข้อมูลเก็บไว้ในฐานข้อมูล A ส่วนเพื่อนของเรามีข้อมูลจัดเก็บอยู่ในฐานข้อมูล B ซึ่งมีรูปแบบและอยู่บนแพลตฟอร์มที่แตกต่างกัน การที่จะเอาข้อมูลจากฐานข้อมูลที่มีรูปแบบต่างกันในอดีตนั้นทำได้ยากหรือบางครั้งก็อาจทำไม่ได้เลยแต่ ณ ตอนนี้นเราสามารถแปลงข้อมูลของเราเป็น XML ซึ่งมีลักษณะคล้ายเท็กซัไฟล์ธรรมดาที่โปรแกรมหรือเครื่องต่าง ๆ สามารถเข้าใจได้ ผนวกกับการที่เรามีไฟล์ DTD กำกับกับการแลกเปลี่ยนข้อมูลทั้งสองฝ่าย แอปพลิเคชันก็จะสามารถใช้นิยามที่กำหนดใน DTD นั้นระบุได้ว่าข้อมูลที่รับส่งกันนั้นมีความถูกต้องหรือไม่

ข้อจำกัดของ DTD

จากที่กล่าวมาดูเหมือนว่า DTD จะเต็มไปด้วยประโยชน์นานับการ แต่อย่างไรก็ดี ตัว DTD เองก็ยังมีข้อจำกัดเหมือนกันตัวอย่างเช่น ตัว DTD เองไม่เข้าใจเนมสเปซ (Namespace) และชนิดของข้อมูลใน DTD นั้นล้วนแล้วแต่เป็น String ทั้งสิ้น ไม่สามารถนิยามชนิดข้อมูลอื่น เช่น Float หรือ Integer รวมทั้งไม่สามารถตรวจสอบขอบเขตของค่าต่าง ๆ ภายในแอดทริบิวต์ได้ เป็นต้น

สคีมา (Schema)

เป็นอีกรูปแบบหนึ่งของการนิยาม เอกสาร XML โดยสคีมา ต่างจาก DTD ตรงที่มีรูปประโยค (Syntax) เป็นของตัวเอง แต่ปัจจุบันนี้สคีมา ยังไม่ได้รับการยอมรับโดย W3C แต่อย่างไรก็ตามความสามารถของสคีมา ไม่ได้ด้อยไปกว่า DTD เลย นอกจากการจัดสร้างข้อมูลที่ DTD นำเสนอแล้วสคีมา ยังช่วยให้สามารถกำหนดชนิดของข้อมูล ใช้เนมสเปซ และกำหนดช่วงค่าของแอดทริบิวต์ และอีลิเมนต์ได้

3.3 Parser

Parser เป็นเหมือนตัวแปลภาษา ซึ่งจะทำหน้าที่ในการแปลเอกสาร XML ให้อยู่ในรูปแบบที่เราสามารถนำมาเรียกใช้ได้ การทำงานของตัว Parser เองจะใช้การแปลหรือวิเคราะห์โครงสร้างใน 2 ลักษณะ อันได้แก่ DOM (Document Object Model) และ SAX (Simple API for XML) ซึ่งเราจะกล่าวถึงราย

ละเอียดของทั้งสองตัวนี้ต่อไป ปกติการเรียกใช้ข้อมูลในแต่ละอีลีเมนต์ของเอกสาร XML จำเป็นต้องผ่าน Parser ดังนั้น Parser จึงเปรียบเสมือนด่านแรกของการใช้งานเอกสาร XML โดย parser จำเป็นต้องตรวจสอบเอกสาร XML ให้ well formed ก่อนแล้วจึงเข้าถึงอีลีเมนต์ต่าง ๆ ภายในเอกสาร XML ตามวิธีการต่าง ๆ ต่อไป

โดยทั่วไปการทำงานของ Parser จำต้อง Parse เอกสาร XML ไปยังแอปพลิเคชันที่ต้องการใช้งานเอกสาร XML นั้น การ Parse เอกสาร XML นั้นหมายถึงการที่เราต้องเขียนโปรแกรมให้แอปพลิเคชันสามารถเข้าใจได้ว่าเอกสาร XML ของเรานั้นมีการจัดโครงสร้างเป็นเช่นไร อันที่จริงเราได้ Parse เอกสารโดยสามัญสำนึกเป็นประจำอยู่แล้ว เช่น ดูว่าในเอกสารประกอบไปด้วยอีลีเมนต์ อะไร ในแต่ละอีลีเมนต์มีแอตทริบิวต์หรือไม่ เป็นต้น แต่สำหรับโปรแกรมแล้วจะมองเห็นเอกสารนั้นเป็นเพียงอักขระธรรมดา ๆ เท่านั้น ด้วยเหตุนี้เราจึงจำเป็นต้องสร้างวิธีการที่จะทำให้แอปพลิเคชัน สามารถเข้าใจถึงโครงสร้างของเอกสาร XML ของเรานั้นก็คือการ Parse เอกสารนั่นเอง และเราจะเรียกสิ่งที่ Parse เอกสารนั้นว่า Parser

ตัวอย่างของ Parser ที่เราพบเห็นได้บ่อยที่สุดคือ Parser ที่มาพร้อมกับโปรแกรม Internet Explorer 5 (IE5) ดังที่เราเคยใช้ IE5 เปิดข้อมูลที่เป็น XML ซึ่งไม่ได้เชื่อมต่อกับ Style Sheet แล้วเห็นข้อมูลในลักษณะที่เป็นข้อมูลโครงสร้างคล้ายทวีวีว นั้นก็เนื่องมาจากในตัวของ IE5 มี Parser ซึ่งใช้การเข้าถึงข้อมูลในแบบของ DOM นั่นเอง

3.4 Document Object Model (DOM)

เป็น Application Programming Interface (API) สำหรับเอกสาร XML ซึ่งได้รับการยอมรับให้เป็นมาตรฐานของ W3C โดยมีเป้าหมายคือต้องการสร้างอินเทอร์เน็ตเฟสที่เป็นมาตรฐาน และใช้กับแอปพลิเคชันได้หลากหลาย ดังนั้น DOM จึงสามารถใช้ร่วมกับภาษาคอมพิวเตอร์ใดก็ได้ บนระบบปฏิบัติการใดก็ได้

การทำงานของ DOM จะเริ่มต้นเมื่อโปรแกรมหรือแอปพลิเคชันไปเรียก Parser ให้ไปโหลดเอกสาร XML เข้าสู่หน่วยความจำของคอมพิวเตอร์ เมื่อเอกสารถูกโหลดเสร็จเรียบร้อยแล้ว DOM ก็จะทำให้โปรแกรมสามารถดึงข้อมูลมาใช้งานหรือดำเนินการต่าง ๆ กับข้อมูลได้

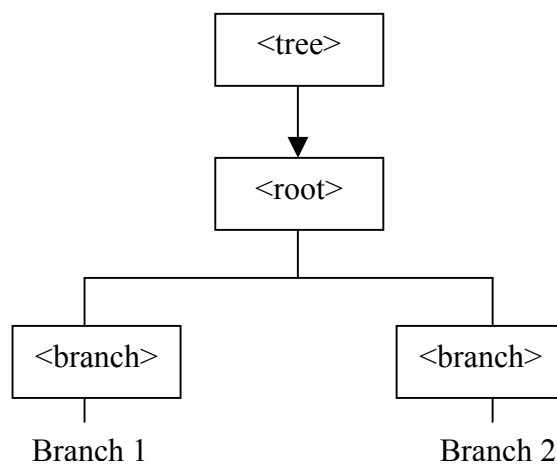
DOM จะมองโครงสร้างเอกสาร XML ในรูปแบบโครงร่างต้นไม้ หรือทรีโดยระดับบนสุดของทรี เราจะเรียกว่า “DocumentElement” ซึ่งอีลีเมนต์ ดังกล่าวจะแตกสาขาออกไปเป็นอีลีเมนต์ย่อย ๆ ตั้งแต่ 1 อีลีเมนต์ขึ้นไป โดยอีลีเมนต์ ย่อยที่แตกออกจาก documentElement เราจะเรียกว่า “โหนดลูก” (childNodes)

```

<tree>
  <root>
    <branch>branch 1</branch>
    <branch>branch 2</branch>
  </root>
</tree>

```

DOM จะมองเอกสารดังกล่าวออกเป็นทรีดังรูปที่ 3-2



รูปที่ 3-2 แสดงแผนภาพแสดงลำดับชั้นการแปลเอกสารของ DOM

ภายใต้การทำงานของโปรแกรม Internet Explorer 5 จะมี XML Parser ฝังอยู่ในตัวอยู่แล้วโดยจะสนับสนุนฟังก์ชันที่จำเป็นทั้งหมดที่ใช้ในการเข้าถึงแต่ละโหนดของทรี ค่าแอดทรีบิวต์ ของโหนด จัดการกับค่าต่าง ๆ ของโหนด อาทิ การเพิ่ม ลบ และการแก้ไข เป็นต้น

จุดเด่นของ DOM

- โปรแกรมจะ Parse เอกสารเพียงครั้งเดียว เมื่อ DOM ได้สร้างทรีขึ้นมาแล้วทรีนั้นก็จะอยู่ในหน่วยความจำตลอดจนกว่าเราจะสั่งยกเลิก และในขณะที่ทรีอยู่ในหน่วยความจำเราก็สามารถเข้าถึงข้อมูลที่อยู่ภายในทรีได้อย่างรวดเร็ว
- ข้อมูลที่สร้างเป็นทรีจะทำให้ข้อมูลเกิดความสัมพันธ์ ทำให้การเรียกใช้งานทำได้ง่าย

จุดด้อยของ DOM

- ในกรณีที่เอกสารมีขนาดใหญ่และซับซ้อนการสร้างทรีจะทำให้ช้า และกินหน่วยความจำมาก
- ผู้พัฒนาโปรแกรมจำเป็นจะต้องเข้าใจถึงโครงสร้างโดยรวมของเอกสารเป็นอย่างดี จึงจะสามารถเขียนโปรแกรมโดยใช้ DOM ได้ กล่าวคือ ในส่วนของการเขียนโค้ดด้วยวิธีการของ DOM นั้นจะค่อนข้างยาวและยุ่งยากอยู่พอสมควร

3.5 Simple API for XML (SAX)

เป็น Application Programming Interface (API) ที่จะมีการทำงานก็ต่อเมื่อมีเหตุการณ์ หรือการร้องขอข้อมูลใด ๆ เกิดขึ้น เรามักเรียก model การทำงานแบบนี้ว่า event-based model กล่าวคือ SAX จะไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนเลยว่าเอกสาร XML นั้นมีโครงสร้างเป็นเช่นไร เมื่อแอปพลิเคชันมีการร้องขอมาจึงจะรายงานเหตุการณ์ของการ Parse ไปยังแอปพลิเคชันดังกล่าว

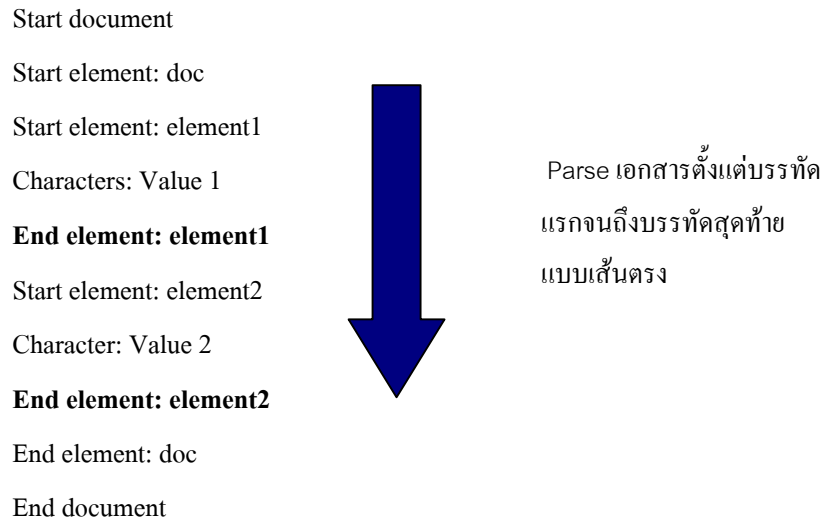
หลักการทำงานของ SAX จะมองเป็นเพียง “จุดเริ่มต้น” กับ “จุดสิ้นสุด” ของเอกสารและของอีลีเมนต์เท่านั้น และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น เมื่อได้สิ่งที่ต้องการแล้วก็จะสิ้นสุดการทำงาน เมื่อเปรียบเทียบกับวิธีการของ DOM ซึ่งจะนำข้อมูลทั้งหมดมาสร้างเป็นทรีทำให้ DOM มีข้อจำกัดในเรื่องของหน่วยความจำซึ่งจะต้องใช้หน่วยความจำมากหากทรีที่สร้างมีขนาดใหญ่มาก ๆ ส่วน SAX นั้นจะเข้าถึงข้อมูลได้ง่ายกว่า การเขียนโปรแกรมทำได้ง่ายกว่าเพราะมองข้อมูลในลักษณะของเส้นตรงคือ จากเริ่มต้นของเอกสาร XML ลงไปเรื่อยจนถึงอีลีเมนต์ ที่เราต้องการ ดังนั้น SAX สามารถ Parse เอกสารที่มีขนาดใหญ่ มากกว่าหน่วยความจำที่เรียกได้เพราะไม่ต้องจำข้อมูลทั้งหมดของเอกสาร จุดนี้เองจึงทำโปรแกรมระบบฐานข้อมูลใหญ่ ๆ หลายบริษัทนิยมใช้วิธีนี้กัน

พิจารณาตัวอย่างเอกสาร XML ในรูปที่ 3-3

```
<?xml version="1.0">
<doc>
  <element1>Value1</element1>
  <element2>Value2</element2>
</doc>
```

รูปที่ 3-3 แสดงตัวอย่างภาษา XML

SAX จะมองเอกสารข้างต้นไปที่ละบรรทัด ๆ ในลักษณะของเหตุการณ์เชิงเส้น (linear events)
ดังรูปที่ 3-4



รูปที่ 3-4 แสดงตัวอย่างการแปลเอกสารในแบบของ SAX

ตัวอย่างการ Parse ข้างต้นเป็นการ Parse เอกสารทั้งเอกสาร ในทางปฏิบัติแล้วเรามักไม่ได้มีการ Parse เอกสารทั้งหมด เมื่อเราได้ข้อมูลที่ต้องการแล้วก็จะหยุด วิธีการ Parse แบบ SAX นี้จึงทำงานได้อย่างรวดเร็วและเขียน โปรแกรมได้ง่ายมาก

จุดเด่นของ SAX

- เข้าถึงข้อมูลได้ง่าย และรวดเร็ว
- เขียนโปรแกรมได้ง่าย เนื่องจากการเข้าถึงข้อมูลเป็นลักษณะเส้นตรง
- ใช้หน่วยความจำน้อย สามารถเข้าถึงข้อมูลหรือเอกสารที่มีขนาดใหญ่กว่าหน่วยความจำที่มีของเครื่องได้

จุดด้อยของ SAX

- ไม่สามารถบอกความสัมพันธ์ของข้อมูลได้
- เป็นวิธีการที่ไม่ค่อยมีประสิทธิภาพหากเป็นการเข้าถึงข้อมูลเดิมบ่อย ๆ เนื่องจากต้องเริ่ม Parse ใหม่ทุกครั้ง

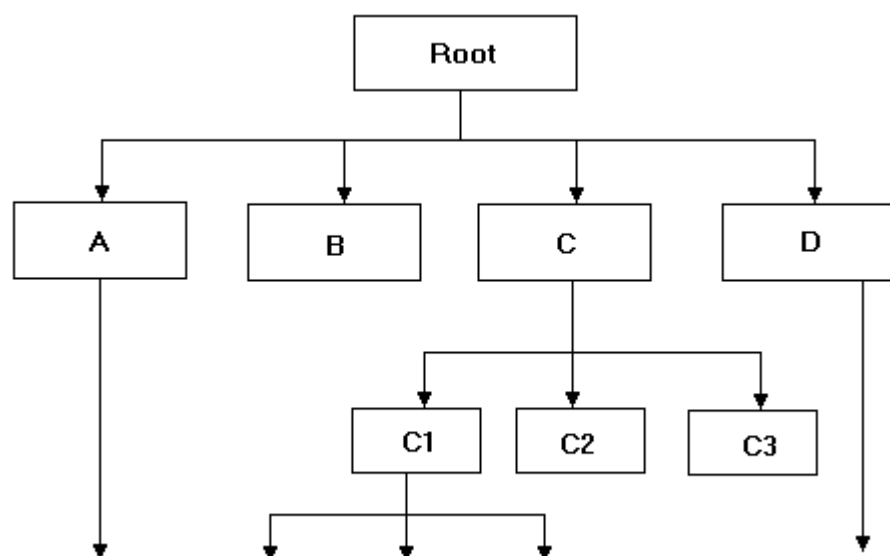
3.6 การเปรียบเทียบการทำงานระหว่าง DOM กับ SAX

ทั้ง DOM และ SAX ต่างก็มีจุดเด่นและจุดด้อยอยู่ในตัว การใช้งานจึงขึ้นอยู่กับดุลพินิจของผู้พัฒนาว่าแอปพลิเคชันที่ใช้เอกสาร XML นั้นมีความเหมาะสมกับวิธีการใดมากที่สุด

```
<?xml version="1.0"?>
<root>
  <A>a</A>
    <B>
      <B1>b1</B1>
      <B2>b2</B2>
    </B>
    <C>c</C>
    <D>
      <D1>d1</D1>
      <D2>d2</D2>
      <D3>d3</D3>
    </D>
</root>
```

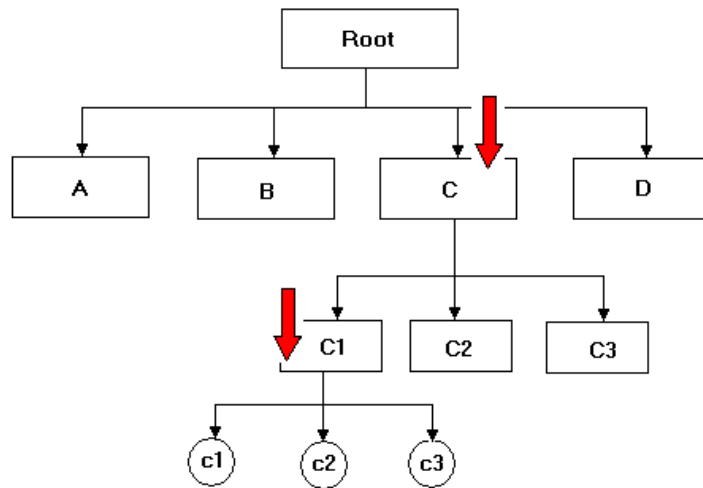
รูปที่ 3-5 แสดงตัวอย่างเอกสาร XML ที่แสดง Tag ต่าง ๆ

จากรูปที่ 3-5 ถ้าเป็นวิธีการของ DOM จะมองเอกสารเป็นทรีที่เกิดจากการประกอบกับของโหนดต่าง ๆ และจะมองอีลีเมนต์แต่ละตัวเป็น Object ซึ่งมุมมองจะออกมาดังรูปที่ 3-6



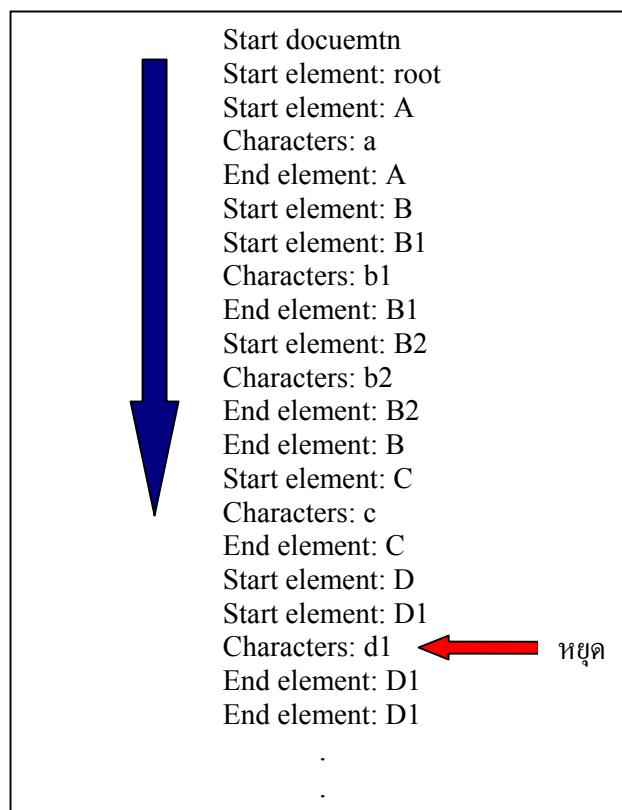
รูปที่ 3-6 แสดงมุมมองการมองเอกสาร XML ในแบบของ DOM

แล้วเมื่อเราต้องการเข้าถึงข้อมูลใด DOM ก็จะใช้วิธีการเข้าไปยังโหนดต่าง ๆ ใน 트리เพื่อหาข้อมูลที่ต้องการ เช่นเราต้องการหาค่าอีลีเมนต์ของ c1 ว่ามีค่าเท่าใด ก็จะมีลักษณะการทำงานดังรูปที่ 3-7



รูปที่ 3-7 แสดงการเข้าถึงค่าภายในอีลีเมนต์ c1

ส่วน SAX จะไม่มีการสร้างมุมมองใด ๆ ของเอกสารไว้เลย การทำงานของ SAX จะเริ่มต้นก็ต่อเมื่อเกิดเหตุการณ์ใด ๆ ขึ้นเท่านั้น เช่นเราต้องการหาอีลีเมนต์ที่ชื่อ d1 SAX ก็จะเริ่มอ่านเอกสารดังรูปที่ 3-8



รูปที่ 3-8 แสดงการ Parse เอกสาร XML ในแบบของ SAX

ทั้ง DOM และ SAX ต่างมีจุดเด่นและจุดด้อยที่แตกต่างกัน ในทางปฏิบัติจึงขึ้นอยู่กับลักษณะการใช้งานว่าจะเหมาะสมกับวิธีการ Parse แบบใด เช่น หากให้ความสำคัญกับความสัมพันธ์ของข้อมูลก็ควรใช้ DOM แต่ถ้าต้องการวิธีการที่ทำให้เขียนโปรแกรมได้ง่ายและไม่เปลืองหน่วยความจำในการประมวลผลก็ควรใช้แบบ SAX เป็นต้น

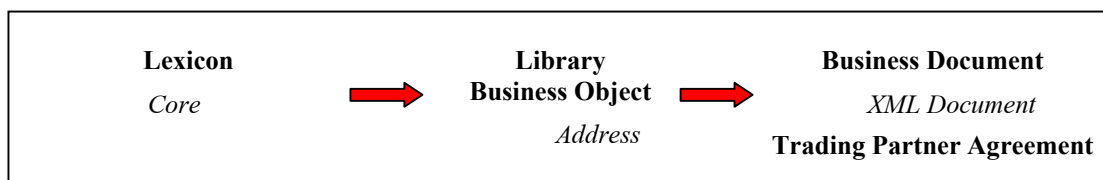
3.7 Electronic Business XML (ebXML)

เป็นมาตรฐานการส่งข้อมูลในรูปแบบของ XML ในรูปแบบที่มีพื้นฐานมาจาก EDI โดยมาตรฐานของ ebXML นี้เป็นส่วนสำคัญที่จะช่วยอธิบายถึงการใช้งานและสร้างแอปพลิเคชัน มาตรฐานนี้ได้รับการเผยแพร่อย่างเป็นทางการในเดือน พฤษภาคม 2543

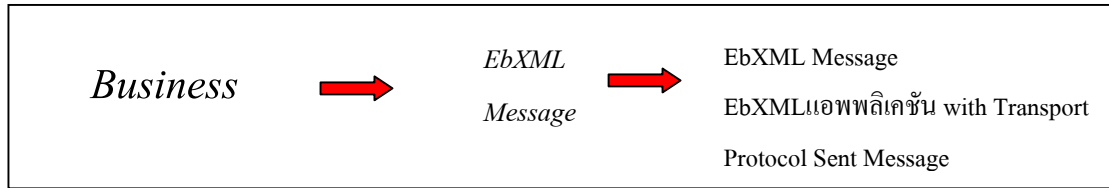
ภายใน ebXML ประกอบไปด้วยการใช้งานมาตรฐานต่าง ๆ อีกมากมาย อาทิเช่น มาตรฐานของ XML, URI, HTTP ฯลฯ โดยมีการส่งข้อมูลในลักษณะของ Message ที่ใกล้เคียงกับการทำงานของ SOAP

การทำงานของ ebXML จะอาศัยฟังก์ชัน 2 ตัว ได้แก่ Business Operational View (BOV) และ Functional Service View (FSV) โดย BOV จะมีหน้าที่ในการนำข้อมูล Business มาแปลงให้เป็นข้อมูลที่อยู่ในรูปของ Business Object จากนั้นก็จะนำ Object ที่ได้มาแปลงเป็นเอกสารที่สามารถใช้งานได้เรียกว่า Business Document

การทำงานในส่วน of FSV จะเป็นการทำงานที่ครอบคลุมถึงการใช้งาน Protocol การเชื่อมต่อข้อมูล และการใช้งานฟังก์ชันต่าง ๆ ที่สนับสนุน BOV ตัวอย่างเช่นเมื่อเบราว์เซอร์ได้รับเอกสาร ebXML ก็จะนำมาผ่านการทำงานของ FSV ซึ่งมีการใช้งานเครื่องมือต่าง ๆ เช่น เครื่องมือที่ใช้แปลง XML ให้อยู่ในรูปแบบของเอกสาร ebXML จากนั้นก็จะเรียกใช้งานโปรโตคอลต่าง ๆ เช่น HTTP เพื่อส่งเอกสาร ebXML นั้นต่อไป ดังรูปที่ 3-9 และรูปที่ 3-10



รูปที่ 3-9 แสดงการทำงานของ Business Operational View



รูปที่ 3-10 แสดงการทำงานของ Function Service View

3.8 XLM Common Business Library (xCBL)

เป็น Library มาตรฐานที่สร้างขึ้นโดยบริษัท Commerce One เพื่อรวบรวมข้อมูลและชื่อต่าง ๆ ที่ถูกออกแบบไว้ใน DTD ซึ่งจะช่วยการออกแบบ DTD ในแอปพลิเคชันประเภท B2B ให้เป็นไปตามแบบมาตรฐานสากล เพื่อให้การใช้งาน XML ร่วมกันระหว่างองค์กรเป็นไปได้อย่างถูกต้องและอยู่บนมาตรฐานเดียวกัน

แต่เนื่องจากขอบเขตความสารถของ DTD เองนั้นมีข้อจำกัดดังที่ได้กล่าวไว้ข้างต้นทำให้การใช้งานไม่ดีเท่าที่ควรทาง Commerce One จึงได้ออกแบบ XML ในรูปแบบของสกีมาให้ด้วย ดังนั้นนอกจากเราจะสามารถใช้งาน xCBL รูปแบบของ DTD เพื่อตรวจสอบเอกสาร XML ที่ valid แล้ว เรายังสามารถใช้ xCBL ในรูปแบบของสกีมา ซึ่งทาง Commerce One เรียกว่า “Schema for Object-Oriented XML” (SOX) เพื่อใช้ในการตรวจสอบเอกสาร XML ที่ valid เช่นกัน

3.9 Commerce XML (cXML)

เช่นเดียวกันกับ xCBL ตัว cXML เองก็เป็น Library ที่เก็บรวบรวมชื่อและรายละเอียดของ XML ในลักษณะของการใช้งานในรูปแบบของ DTD แต่ต่างกันตรงที่ cXML ถูกพัฒนาขึ้นโดยบริษัท Ariba Technologies, Inc. ซึ่งเป็นบริษัทที่รับพัฒนา XML และแอปพลิเคชันให้กับบริษัทใหญ่ ๆ มากมาย อาทิเช่น Microsoft, Cisco Systems, Hewlett Packard ฯลฯ ทำให้ cXML ได้รับการยอมรับให้ใช้งานได้อย่างกว้างขวาง แต่ตัว cXML ไม่มีเวอร์ชันที่ออกแบบให้เป็นสกีมา เช่นเดียวกับ xCBL

การใช้งาน xCML มีทั้งการใช้งานในผลิตภัณฑ์ที่ต้องซื้อจาก Ariba และมีทั้งเวอร์ชันที่แจกให้ใช้ฟรี จุดสำคัญอีกประการหนึ่งของ cXML เองคือสามารถใช้งานร่วมกับภาษา Active Server Pages (ASP) ได้ง่าย แต่ cXML จะไม่สามารถใช้เนมสเปซได้ (เนื่องจากข้อจำกัดของ DTD)